Event Loop

The loop that decides what runs next on one thread

1 Call stack sync code to the end → **2 Microtasks** run all of them → **3 Macrotask** run one, then repeat

	Micro	Macro
Examples	then, await, queueMicrotask	setTimeout, events
Runs	after stack empties	one per turn
Repaint	none between them	possible after each

1 Timers are late

```
setTimeout(tick, 0);
heavy();   // takes 2 s
console.log("sync");
// order: heavy, sync, tick
```

2 Microtasks can starve

```
function loop() {
  Promise.resolve().then(loop);
}
loop();   // timers never run
```

TRAP What does this log?

```
console.log("A");
setTimeout(() => console.log("B"));
queueMicrotask(() => console.log("C"));
console.log("D");
```

A D C B

Sync first, then microtasks, then timers.

KEY TAKEAWAY

- `stack` Runs to the end first
- `micro` All run before any timer
- `macro` One per loop turn

? Follow-up questions to expect

- **When does code after `await` run?** → As a microtask, once the stack is empty.
- **Why does `setTimeout(f, 0)` run late?** → Stack, then microtasks. Nested timers: ≥ 4 ms.
- **Is JavaScript single-threaded?** → Yes: one call stack. The browser does the waiting.
- **Why can a heavy loop freeze the page?** → It blocks the stack: no clicks, no repaint.